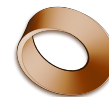


Name: \_\_\_\_\_

Class: \_\_\_\_\_

Mr. Rawson • GCSE Computer Science



Year 10 2026

---

# Data Representation 2

## Characters, Images, Sound & Compression

OCR J277 • 1.2.4 Data Storage • 1.2.5 Compression

---

01001000 01101001

72 105    ``Hi''

**In this booklet:** characters, character sets, ASCII and Unicode  
images as pixels • colour depth • resolution  
metadata  
sound • sampling • sample rate • bit depth  
compression • lossy and lossless

## How this booklet works

Booklet 1 got you as far as binary shifts, and the quiz closed that off. Everything in *this* booklet is a different kind of question. Booklet 1 asked *how do we write a number in binary?* This one asks *how do we write a **letter** in binary? A **photograph**? A **song**?*

The answer is always the same, and it is worth having in your head before you start: there is only one kind of data in a computer, and it is numbers. So the real question every time is **what number do we choose, and what do we agree it means?**

### Key concept

**This booklet is built to be looked things up in, not just worked through.**

Every idea in here is laid out the same way, in the same order, every time:

1. **The rule** — stated outright, in a blue box like this one. You are never expected to work a rule out for yourself before you have been told it.
2. **A worked example** — in an amber box, done in full with real numbers.
3. **What just happened** — in plain English underneath, explaining the example one step at a time.
4. **The trap** — in a red box. The mistake nearly everybody makes, named *before* you make it.

**Only then is there anything for you to do.** If you are stuck on a practice question, the rule you need is on the same page or the one before it. Look it up. That is not cheating — it is what the booklet is for.

**Where this ends up.** Everything here is on the **end-of-unit test**, together with the numbers work from Booklet 1.

### What you will be able to do by the end:

| Spec  | Skill                                                                           |
|-------|---------------------------------------------------------------------------------|
| 1.2.4 | Explain how a computer stores a letter, and what a <i>character set</i> is      |
| 1.2.4 | Work out how many characters a given number of bits can represent               |
| 1.2.4 | Explain the difference between ASCII and Unicode, and what each costs           |
| 1.2.4 | Explain how an image is stored as pixels, and what <i>metadata</i> is           |
| 1.2.4 | Calculate an image file size, and say what colour depth and resolution do to it |
| 1.2.4 | Explain how sound is sampled, and calculate a sound file size                   |
| 1.2.5 | Explain why we compress, and choose between lossy and lossless with a reason    |

## Characters — storing a letter

A computer cannot store the letter A. It can only store numbers. So we do the only thing available: we **agree on a number** for every character, write that number down in binary, and everybody uses the same list.

### Key concept

**A character set is an agreed list that gives every character a number.**

That is the entire idea. The number for a character is called its **character code**.

Nothing about the letter A *makes* it 65. Somebody decided, everybody agreed, and now it is 65 everywhere. A character set is an **agreement**, not a discovery.

**Here is the list.** These are the 95 printable ASCII characters and their codes. You are **not** expected to learn it — it is here to look things up in, and any exam question that needs a code will give you the code.

| Dec Chr | Dec Chr | Dec Chr | Dec Chr | Dec Chr |
|---------|---------|---------|---------|---------|
| 32 _    | 51 3    | 70 F    | 89 Y    | 108 l   |
| 33 !    | 52 4    | 71 G    | 90 Z    | 109 m   |
| 34 "    | 53 5    | 72 H    | 91 [    | 110 n   |
| 35 #    | 54 6    | 73 I    | 92 \    | 111 o   |
| 36 \$   | 55 7    | 74 J    | 93 ]    | 112 p   |
| 37 %    | 56 8    | 75 K    | 94 ^    | 113 q   |
| 38 &    | 57 9    | 76 L    | 95 _    | 114 r   |
| 39 '    | 58 :    | 77 M    | 96 `    | 115 s   |
| 40 (    | 59 ;    | 78 N    | 97 a    | 116 t   |
| 41 )    | 60 <    | 79 O    | 98 b    | 117 u   |
| 42 *    | 61 =    | 80 P    | 99 c    | 118 v   |
| 43 +    | 62 >    | 81 Q    | 100 d   | 119 w   |
| 44 ,    | 63 ?    | 82 R    | 101 e   | 120 x   |
| 45 -    | 64 @    | 83 S    | 102 f   | 121 y   |
| 46 .    | 65 A    | 84 T    | 103 g   | 122 z   |
| 47 /    | 66 B    | 85 U    | 104 h   | 123 {   |
| 48 0    | 67 C    | 86 V    | 105 i   | 124     |
| 49 1    | 68 D    | 87 W    | 106 j   | 125 }   |
| 50 2    | 69 E    | 88 X    | 107 k   | 126 ~   |

**Read down the columns and you can see the ordering rule before it is even stated.** The capitals run 65–90 with no gaps, the lowercase run 97–122, and the digits 0–9 sit at 48–57. **Codes 0–31 are missing from this table on purpose** — they are *control codes* rather than symbols, and they print nothing. 10 is a new line, 9 is a tab.

### Worked example

Storing the word Hi.

| Character | Code (denary) | In 8-bit binary |
|-----------|---------------|-----------------|
| H         | 72            | 01001000        |
| i         | 105           | 01101001        |

So the word Hi is stored as 01001000 01101001 — 16 bits, 2 bytes.

**What just happened, step by step.** We looked H up in the agreed list and found 72. We converted 72 to binary exactly the way Booklet 1 taught ( $64 + 8 = 72$ ). We did the same for i, which is 105. Then we wrote the two bytes one after the other. **That is all “storing text” means** — one byte per character, in order.

### The trap

**H and h are two different characters, with two different codes.** Capitals and lowercase are not the same thing to a computer. H is 72; h is 104.

**And the character ‘7’ is not the number 7.** The *digit* 7 typed on a keyboard has character code 55. The *number* 7 is stored as 00000111. They look the same on screen and are completely different to the machine — which is exactly why you had to use `int()` in programming to turn one into the other.

**Practice 3.1** — Using the table above and the ordering rule on page 5, write the character codes, in denary, for the word Cat.

C = \_\_\_\_\_ a = \_\_\_\_\_ t = \_\_\_\_\_

## ASCII, and the one fact you must know

**ASCII** is the character set almost everything has used since the 1960s. It is the list that says A is 65.

### Key concept

**ASCII uses 7 bits, which gives  $2^7 = 128$  different characters.**

In the exam, **ASCII is always written in 8 bits** — 7 bits of code with a 0 on the front. So A is 01000001.

128 characters is enough for: the English alphabet in capitals and lowercase, the digits 0–9, punctuation, and some control codes. It is not enough for anything else.

**The landmarks.** You do **not** have to memorise ASCII — the specification says so, and any exam question that needs a code will give it to you. These four are worth knowing anyway, because they make everything else quick:

| Codes   | Characters |                                   |
|---------|------------|-----------------------------------|
| 32      | space      | a space is a character too        |
| 48--57  | 0 to 9     | so the digit 7 is $48 + 7 = 55$   |
| 65--90  | A to Z     | 26 in a row, no gaps              |
| 97--122 | a to z     | exactly 32 more than the capitals |

### Key concept

**THE examinable fact: a character set is in order.**

The code for B is one more than the code for A. The code for C is one more than B. All the way to Z.

This is the thing the exam actually tests — not the numbers themselves.

### Worked example

**You are told A is 65. What is F?**

F is the 6th letter. A is the 1st. So F is 5 letters after A.

$$65 + 5 = \mathbf{70}$$

**What just happened.** We never looked F up. We counted how far it is from a letter we *were* given, and added that on. Because the set is in order, counting works — and that is why you can answer these questions with one fact instead of 128.

### The trap

**Count the gaps, not the letters.** From A to F there are six letters but only **five steps**. Writing  $65 + 6 = 71$  is the single most common mistake here, and 71 is G.

Check it on a short one you can see: A=65, so B must be 66 — one letter along, one step, +1. Not +2.

**Practice 4.1** — a is 97. What is e? \_\_\_\_\_

**Practice 4.2** — P is 80. What is M? \_\_\_\_\_ *(Careful — this one goes backwards.)*

## How many characters can you fit?

This is the same arithmetic as colour depth later in the booklet, so it is worth getting solid once.

### Key concept

**With  $n$  bits you can represent  $2^n$  different characters.**

Each extra bit **doubles** how many things you can tell apart — because each new bit can be 0 or 1, giving two versions of everything you already had.

| Bits | Characters ( $2^n$ ) |                  |
|------|----------------------|------------------|
| 1    | 2                    | just two symbols |
| 2    | 4                    |                  |
| 3    | 8                    |                  |
| 4    | 16                   | one nibble       |
| 7    | 128                  | <b>ASCII</b>     |
| 8    | 256                  | one byte         |
| 16   | 65,536               |                  |

### Worked example

**A character set needs to hold 200 different characters. How many bits per character?**

Try them in order:

7 bits gives  $2^7 = 128$ . Not enough — 128 is less than 200.

8 bits gives  $2^8 = 256$ . Enough, with room spare.

**Answer: 8 bits.**

**What just happened.** We did not calculate anything clever. We worked up the powers of 2 until one was big enough, and took the first one that was. **“Big enough” is the test** — 256 is more than we need, and that is fine. You cannot use 7.5 bits.

### The trap

**Bits go up in ones; the number of characters doubles.** Going from 7 bits to 8 bits does not add one character, or add 7 — it goes from 128 to 256. Adding a bit doubles.

**Practice 5.1** — How many characters can 5 bits represent? \_\_\_\_\_

**Practice 5.2** — A set must hold 40 characters. What is the smallest number of bits? \_\_\_\_\_

*Hint for 5.2: 5 bits gives 32, which is not enough.*

## Unicode — when 128 is not enough

ASCII was designed in America for English. It has no é, no ü, no Greek, no Arabic, no Chinese, and no emoji. With 7 bits there is simply no room.

### Key concept

**Unicode is a much larger character set, built to hold every writing system in the world.**

It uses **more bits per character** — commonly 16, and up to 32 — so it can represent over a million different characters.

**Unicode keeps the ASCII codes the same.** A is still 65. That was deliberate, so that old text did not break.

**The trade-off, which is what the exam asks about:**

|                     | ASCII             | Unicode                 |
|---------------------|-------------------|-------------------------|
| Bits per character  | 7 (8 in the exam) | 16 or more              |
| How many characters | 128               | over 1,000,000          |
| Languages covered   | English only      | effectively all of them |
| File size           | <b>smaller</b>    | <b>larger</b>           |

### Worked example

**The word Hi stored two ways.**

In ASCII at 8 bits per character:  $2 \times 8 = \mathbf{16 \text{ bits}}$  (2 bytes).

In Unicode at 16 bits per character:  $2 \times 16 = \mathbf{32 \text{ bits}}$  (4 bytes).

Same two letters. **Twice the storage.**

**What just happened, and why it matters.** Nothing about the word changed — only the agreement about how many bits each character gets. Unicode's advantage is that it can store *any* character; the price is that *every* character costs more, including the plain English ones that did not need it. **That sentence is the answer to almost every Unicode exam question.**

### The trap

**“Unicode is better” is not an answer.** Every exam question here wants a **trade-off**: what you gain *and* what it costs. Gain — far more characters, every language. Cost — bigger files.

**Practice 6.1** — Give one advantage and one disadvantage of using Unicode instead of ASCII.

## Images — a picture is a grid of numbers

Same problem as letters, same solution. A computer cannot store a photograph. It stores numbers and agrees what they mean.

### Key concept

**An image is split into a grid of tiny squares called pixels. Each pixel is one solid colour, stored as a binary number.**

The picture is just the list of those numbers, read in order — left to right, top to bottom. This kind of image is called a bitmap.

### Worked example

**A tiny black-and-white image, 8 pixels by 8.**

With only two colours we need only **1 bit per pixel**: 0 for white, 1 for black.

```
00111100 row 1
01000010 row 2
10100101 row 3
10000001 row 4
```

That is 4 rows of 8 pixels = 32 pixels = 32 bits so far.

**What just happened.** Each 1 is a black square and each 0 is a white one. Read row 1 — white, white, black, black, black, black, white, white — and you are drawing. **The image and the binary are the same thing**, written two ways. Nothing is hidden.

### The trap

**The grid is not stored.** The computer does not store “8 across, 8 down” as part of the picture data — that is stored separately, as *metadata* (page 10). Without it, the machine has a string of bits and no idea what shape to fold it into.

**Practice 7.1** — Shade the squares for the row 01100110, where 1 is black.

|  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|

## Colour depth — how many colours are possible

### Key concept

**Colour depth** is the number of bits used for each pixel.

With  $n$  bits per pixel you can have  $2^n$  different colours — *exactly* the same arithmetic as characters on page 5.

| Colour depth | Colours ( $2^n$ ) |                             |
|--------------|-------------------|-----------------------------|
| 1 bit        | 2                 | black and white only        |
| 2 bits       | 4                 |                             |
| 4 bits       | 16                |                             |
| 8 bits       | 256               |                             |
| 24 bits      | 16,777,216        | “true colour” — photographs |

### Worked example

**An image uses 4 bits per pixel. How many colours can it show?**

$2^4 = 16$  colours.

**And the other direction:** an image needs 256 colours. What colour depth?

$2^8 = 256$ , so **8 bits** per pixel.

**What just happened.** Both questions are the same doubling table read in opposite directions. Given the bits, raise 2 to that power. Given the colours, ask what power of 2 makes it. **If you can do page 5, you can already do this** — only the word changed.

### The trap

**Colour depth is per *pixel*, not per image.** A 24-bit image does not use 24 bits in total — it uses 24 bits for **every single pixel**. A one-megapixel photo at 24-bit uses 24 million bits.

**Practice 8.1** — How many colours at a colour depth of 5 bits? \_\_\_\_\_

**Practice 8.2** — An image must show 16 colours. What is the smallest colour depth? \_\_\_\_\_

## Resolution, and working out the file size

### Key concept

**Resolution** is how many pixels the image contains — its width times its height.

More pixels means more detail, and more data.

**The formula, and it is the one to memorise:**

$$\text{file size (bits)} = \text{width} \times \text{height} \times \text{colour depth}$$

### Worked example

**An image is 800 pixels wide, 600 pixels high, with a colour depth of 8 bits.**

**Step 1 — how many pixels?**

$$800 \times 600 = 480,000 \text{ pixels.}$$

**Step 2 — multiply by the colour depth.**

$$480,000 \times 8 = 3,840,000 \text{ bits.}$$

**Step 3 — convert to bytes** (divide by 8).

$$3,840,000 \div 8 = 480,000 \text{ bytes.}$$

**Step 4 — convert to kilobytes** (divide by 1,000).

$$480,000 \div 1,000 = \mathbf{480 \text{ kB}}$$

**What just happened, one step at a time.** Step 1 counts the squares. Step 2 says how much each square costs. Steps 3 and 4 are just the unit conversions from Booklet 1 — 8 bits to a byte, 1,000 bytes to a kilobyte. **The physics is all in steps 1 and 2;** the rest is tidying up.

### The trap

**The formula gives you BITS. Almost every exam question wants bytes or kilobytes.** Dividing by 8 is where the marks are lost — not in the multiplication.

Write the unit next to every line of your working. If your answer to “how many kilobytes” is in the millions, you have skipped a division.

**Practice 9.1** — An image is  $100 \times 50$  pixels with a colour depth of 4 bits. Give the file size in **bytes**.

## Metadata — the data about the data

### Key concept

**Metadata is the extra information stored *alongside* an image so that it can be displayed correctly.**

Without it the file is a meaningless string of bits. The computer would not know how wide the picture is, so it could not know where one row ends and the next begins.

Commonly stored: **width, height, colour depth**, file type, date taken, camera settings, sometimes GPS location.

**Why this matters, concretely.** Take the 8-pixel row from page 7 and imagine 64 bits with no width recorded. Is it 8 rows of 8? 4 rows of 16? 2 rows of 32? Every one of those is a different picture from identical data. **Metadata is what makes the bits mean one thing rather than another.**

### The trap

**Metadata makes the file slightly bigger.** If a question asks for a file size “including metadata”, it will give you the metadata size to add on — but you must remember to add it.

## Quality against size — the trade-off

Increase either resolution or colour depth and you get a better image and a bigger file. That sentence is the whole of this section, and the exam asks it constantly.

| If you...                 | Quality                  | File size |
|---------------------------|--------------------------|-----------|
| increase the resolution   | more detail, sharper     | bigger    |
| decrease the resolution   | blocky, pixelated        | smaller   |
| increase the colour depth | smoother, more realistic | bigger    |
| decrease the colour depth | banded, fewer shades     | smaller   |

### Worked example

**Why a photograph is 24-bit and an icon is not.**

A photo of a face needs thousands of near-identical skin tones, so it needs a high colour depth — 24-bit, giving 16.7 million colours. A flat two-colour icon needs 1 bit and would look identical at 24.

**Same rule, opposite answers, because the picture is different.**

**Practice 10.1** — Give **two** pieces of metadata stored with an image.

1. \_\_\_\_\_
2. \_\_\_\_\_

## Sound — turning a wave into numbers

Sound in the real world is a wave: it changes smoothly and never jumps. Computers cannot store “smooth”. So we measure the wave at regular moments and store the measurements.

### Key concept

**Sampling means measuring the height of the sound wave at regular intervals and storing each measurement as a binary number.**

Play the numbers back in order, fast enough, and the ear hears the original sound.

**A sample is never perfect.** It is the wave’s height at one instant, rounded to the nearest value we can store. What happened between two samples is lost for ever.

### Worked example

**Sampling a wave eight times.**

Suppose we measure the wave’s height and get, in order:

3, 5, 7, 6, 4, 2, 1, 2

Those eight numbers, written in binary, *are* the sound file.

**What just happened.** We replaced a smooth curve with eight numbers. Joining those eight points gives a jagged staircase, not a curve — close to the original, but not the original. **Measure more often and the staircase gets finer**, which is exactly what sample rate means on the next page.

### Key concept

**The three things that describe a sound file:**

**Sample rate** how many samples are taken **per second**, measured in **hertz (Hz)**

**Bit depth** how many **bits are used for each sample**

**Duration** how many **seconds** of audio the file holds

### The trap

**Sample rate and bit depth are not the same thing, and they do different jobs.**

**Sample rate** is *how often* you measure — along the time axis.

**Bit depth** is *how precisely* you record each measurement — how many different heights you can tell apart.

More samples, measured more coarsely, is not the same as fewer samples measured finely.

## Working out a sound file size

### Key concept

#### The formula:

$$\text{file size (bits)} = \text{sample rate} \times \text{duration} \times \text{bit depth}$$

Sample rate in Hz, duration in **seconds**, bit depth in bits.

### Worked example

**A recording is sampled at 8,000 Hz for 30 seconds with a bit depth of 8 bits.**

#### Step 1 — how many samples altogether?

$$8,000 \times 30 = 240,000 \text{ samples.}$$

#### Step 2 — multiply by the bit depth.

$$240,000 \times 8 = 1,920,000 \text{ bits.}$$

#### Step 3 — convert to bytes.

$$1,920,000 \div 8 = 240,000 \text{ bytes.}$$

#### Step 4 — convert to kilobytes.

$$240,000 \div 1,000 = \mathbf{240 \text{ kB}}$$

**What just happened.** Step 1 counts how many measurements exist. Step 2 says what each one costs. Steps 3 and 4 are unit conversion. **It is the identical four-step shape as the image calculation on page 9** — count the things, multiply by the cost of one, then tidy the units. Learn one, and you have both.

### The trap

**Duration must be in seconds.** A question saying “3 minutes” is asking you to use 180, not 3. Convert it first, on its own line, before you touch the formula.

**Practice 12.1** — A track is sampled at 44,100 Hz for **3 minutes** at a bit depth of 16 bits. Give the file size in **megabytes**.

### Worked example

#### Quality against size, again.

Raise the **sample rate** — more measurements per second, closer to the original wave, better quality, bigger file.

Raise the **bit depth** — each measurement more precise, better quality, bigger file.

**Both improve quality. Both cost space. There is no free option.**

## Compression — why bother

You have now calculated two file sizes and both were large. Three minutes of CD-quality audio came to nearly 16 MB. A phone photograph is bigger still. Compression is how any of this fits on a device or travels over a network.

### Key concept

**Compression means making a file smaller.**

Why we do it:

- files take up **less storage space**
- they **download and upload faster**
- they use **less bandwidth**, which matters on a mobile connection
- email and messaging services impose **size limits**

### Key concept

**There are exactly two kinds, and the difference is whether anything is thrown away.**

**Lossless** — **nothing is lost.** The file is rewritten more efficiently, and decompressing gives you back the original *exactly*, bit for bit.

**Lossy** — **data is permanently removed.** Detail the human eye or ear is unlikely to notice is discarded. You can never get it back.

**The way to keep them straight.** *Lossy* loses. *Lossless* loses less — in fact it loses nothing. If a question asks whether the original can be recovered perfectly, it is asking you which of these two you chose.

|                       | Lossless                      | Lossy                             |
|-----------------------|-------------------------------|-----------------------------------|
| Original recoverable? | <b>Yes, exactly</b>           | <b>No, never</b>                  |
| File size reduction   | smaller                       | <b>much smaller</b>               |
| Quality               | unchanged                     | reduced                           |
| Typical use           | text, code, spreadsheets, ZIP | photos (JPEG), music (MP3), video |

### The trap

**“Lossy is worse” is not the answer.** Lossy is the right choice for a photo on a website — nobody can see the difference and the page loads far faster.

**But lossy on a program, a spreadsheet or a document is a disaster.** Throw away “a few unnoticeable bits” of program code and it no longer runs. **The right answer always depends on the file.**

## How lossless compression actually works

You are not required to perform a compression algorithm in the exam. But seeing one makes “rewritten more efficiently” stop being a vague phrase.

### Key concept

**Run-length encoding** replaces a run of repeated characters with **a count and the character**.

### Worked example

**Compressing AAAABBBCCD.**

Four As → 4A

Three Bs → 3B

Two Cs → 2C

One D → 1D

AAAABBBCCD (10 characters) becomes 4A3B2C1D (**8 characters**).

And it is **lossless** — reading 4A3B2C1D back gives AAAABBBCCD exactly.

**What just happened.** We did not delete anything. We wrote the same information a shorter way, and the instructions for rebuilding it are complete. **That is precisely what lossless means** — and you can prove it by decompressing and comparing.

### The trap

**Compression can make a file BIGGER, and this is the best question on the topic.**

Try ABABAB — six characters, no runs at all:

ABABAB → 1A1B1A1B1A1B

Six characters have become **twelve**. Every single run was length one, so we stored a count of 1 for every character and doubled the file.

**Run-length encoding only helps when there are long runs.** A big block of sky compresses beautifully. A checkerboard does the opposite.

**Practice 14.1** — Compress WWWWWWWWWWWBWWWWWWWWWWBBB using run-length encoding.

*28 characters become 10 — this one is mostly sky.*

**Practice 14.2** — A student says “compression always makes a file smaller”. Explain why they are wrong.

## Reference — the cheat sheet

**The two formulas. Both give BITS. Divide by 8 for bytes.**

image file size = width × height × colour depth

sound file size = sample rate × duration × bit depth

**The doubling table — characters, and colours. Same table, two topics.**

| Bits | $2^n$ | Bits | $2^n$      |             |
|------|-------|------|------------|-------------|
| 1    | 2     | 8    | 256        | one byte    |
| 2    | 4     | 16   | 65,536     |             |
| 4    | 16    | 24   | 16,777,216 | true colour |
| 7    | 128   |      |            | ASCII       |

**ASCII landmarks.** You do not have to memorise these.

32 space    48--57 digits 0--9    65--90 A--Z    97--122 a--z

**And the fact that IS examined:** a character set is **in order**. B is one more than A.

**Unit conversions, from Booklet 1.**

8 bits = 1 byte    1,000 bytes = 1 kB    1,000 kB = 1 MB    1,000 MB = 1 GB

**The four-step shape for every file-size question:**

1. count the things    2. multiply by the cost of one    3. bits → bytes    4. bytes → kB

**Compression, in one line each.**

**Lossless** — nothing lost, original recovered exactly, smaller.

**Lossy** — data permanently removed, cannot be recovered, much smaller.

**Quality against size.** Raising resolution, colour depth, sample rate or bit depth *always* improves quality and *always* increases file size. There is no free option.

| Term                | Meaning                                                                     |
|---------------------|-----------------------------------------------------------------------------|
| character set       | an agreed list giving every character a number                              |
| character code      | the number that represents one character                                    |
| ASCII               | a 7-bit character set holding 128 characters; written in 8 bits in the exam |
| Unicode             | a much larger character set covering all writing systems; 16 bits or more   |
| pixel               | one square of an image, storing a single colour                             |
| bitmap              | an image stored as a grid of pixels                                         |
| colour depth        | the number of bits used for each pixel                                      |
| resolution          | the number of pixels in an image — width $\times$ height                    |
| metadata            | data stored about the file, such as width, height and colour depth          |
| sampling            | measuring a sound wave at regular intervals and storing each measurement    |
| sample rate         | how many samples are taken per second, in hertz (Hz)                        |
| bit depth           | the number of bits used for each sample                                     |
| duration            | the length of a sound file in seconds                                       |
| compression         | making a file smaller                                                       |
| lossless            | compression where nothing is lost and the original is recovered exactly     |
| lossy               | compression where data is permanently removed                               |
| run-length encoding | a lossless method storing a count and a character for each run              |

## Can I ...?

Tick a column for each skill. Be honest — the point is to find what to practise before the end-of-unit test, not to collect ticks.

| Skill                                                  | Not yet | Getting there | Confident |
|--------------------------------------------------------|---------|---------------|-----------|
| Explain what a character set is, and why A is 65       |         |               |           |
| Work out a character code using the ordering rule      |         |               |           |
| Say how many characters $n$ bits can represent         |         |               |           |
| Give an advantage <i>and</i> a disadvantage of Unicode |         |               |           |
| Explain how an image is stored as pixels               |         |               |           |
| Say what colour depth and resolution each do           |         |               |           |
| Calculate an image file size, in bytes                 |         |               |           |
| Say what metadata is and give two examples             |         |               |           |
| Explain how sound is sampled                           |         |               |           |
| Calculate a sound file size, in bytes                  |         |               |           |
| Explain the difference between lossy and lossless      |         |               |           |
| Choose a compression type and justify the choice       |         |               |           |

**The one thing I most need to practise:**